MediTrack — PostgreSQL + Prisma Roadmap

Codex-ready, phase-by-phase build plan. Web app: React (Vite) + Node/Express + Python OCR microservice, database migrated from MySQL/Sequelize to **PostgreSQL + Prisma ORM**.

Updated Tech Stack

Layer	Technology
Frontend	React.js (Vite) + React Router
Backend (core)	Node.js + Express — auth, scheduling, notifications, AI orchestration
OCR Microservice	Python (FastAPI/Flask) — OpenCV preprocessing + PaddleOCR (benchmark-selected)
Database	PostgreSQL
ORM	Prisma (schema.prisma + Prisma Client + Prisma Migrate)
Cache	Redis — session/login-token storage (not a DB table)
Real-time	Socket.io — live caregiver alerts, escalation feed
Cloud Storage	AWS S3 or Azure Blob Storage — prescription images
AI	Gemini API (called from Node backend) — chat, insights, schedule parsing
Auth	Custom JWT + bcrypt (no Passport.js) + Google Sign-In via google-auth-library
Authorization	Multi-role via relationship links (not fixed account roles)

Why Prisma: Prisma's schema-first workflow (schema.prisma) generates a fully typed client, handles migrations (prisma migrate dev), and its relation syntax maps cleanly onto this schema's heavy use of self-referencing foreign keys (e.g. caregiver_patient_links pointing twice into users). It is also easier for an AI coding agent to work from, since the schema file is the single source of truth for types, relations, and constraints.

Setup commands: npm install prisma @prisma/client → npx prisma init → paste schema.prisma below → set DATABASE_URL in .env → npx prisma migrate dev --name init → npx prisma generate

Authentication Strategy (Decision Record)

Chosen approach: custom JWT + bcrypt, with Google Sign-In as an additional entry point. Passport.js is NOT used in this project.

Passport.js is a middleware framework of pluggable strategies rather than a competing alternative to JWT + bcrypt — a Passport setup would still use bcrypt for local login and JWTs for stateless sessions. Its default model is session-based (serialize/deserialize into a server session), which suits server-rendered apps more than a decoupled React SPA talking to a REST API. Running it stateless (session: false + passport-jwt) turns it into a thin wrapper over the same logic, adding strategy-config indirection without much benefit for a single first-party login method.

For MediTrack — a stateless SPA + API with Redis-backed refresh tokens and permissions derived per relationship rather than per account — explicit custom middleware is simpler to reason about, easier to debug, and more predictable for an AI coding agent to implement. Google Sign-In is added directly via google-auth-library with no Passport dependency.

Key design point: Google Sign-In does not create a parallel auth system. It is one extra endpoint (POST /api/auth/google) that verifies a Google ID token, resolves or creates the User row, and then issues the exact same app-issued access + refresh JWT pair as email/password login. Every downstream route, middleware, and permission check in Phases 2-12 is therefore completely auth-method-agnostic.

OCR Engine Selection (Decision Record — ALREADY COMPLETED)

Chosen engine: PaddleOCR. This comparison has already been run and is closed — Phase 4 implements PaddleOCR only, with no engine-selection logic and no re-benchmarking.

Benchmark run 2026-09-14 on 15 synthetic computer-printed prescription slips (5 layout families, 3 fonts, 2 clean baselines + 13 deliberately degraded with rotation, Gaussian noise, focus blur, a brightness/contrast gradient and JPEG compression). All 45 image/engine runs completed. Every engine received identical preprocessed pixels (grayscale, denoise, pixel-based deskew, CLAHE, adaptive binarization).

Engine	CER %	WER %	Field %	Exact %	Conf.	Latency s
tesseract	1.54	4.20	98.16	97.84	0.915	0.358
easyocr	2.02	8.66	86.68	85.83	0.848	8.526
paddleocr (selected)	0.10	0.62	99.31	99.13	0.989	6.112

Rationale: PaddleOCR's 0.10% mean CER beats the runner-up (Tesseract, 1.54%) by 1.44 percentage points, exceeding the predefined 1.0-point practical tie margin, so the primary metric gives a clear winner. It also leads on field recovery (99.31% vs 98.16%) and is far more consistent across degraded images — Tesseract spiked to 6.55% CER on rx_04 and 5.71% on rx_08, while PaddleOCR stayed near zero throughout. The trade-off accepted: ~6.1 s CPU latency per image versus Tesseract's ~0.36 s. That is acceptable here because scanning is an asynchronous, one-off action per prescription, not a hot path — which is why Phase 4 is built async with a Socket.io completion event from the start.

Stated limitations (carry these into the README, don't overclaim): the test set is synthetic printed English text with controlled layouts and no handwriting, paper folds, or severe perspective distortion. The regex extractor was written for these documented layout families, so field recovery does not establish generalization to unseen clinical layouts. This is development evidence, not clinical validation. Consequently, mandatory human review of every extracted scan is a product requirement in Phase 4, not an optional nicety.

Complete schema.prisma (all 20 models)

Feed this file directly into the project as prisma/schema.prisma. Every table, field, type, enum, and foreign-key relationship from the original design is represented — nothing needs to be re-derived.

```
// ============================================================
// MediTrack — Prisma Schema (PostgreSQL)
// Generated to match the original relational schema exactly.
// Run: npx prisma migrate dev --name init
// ============================================================

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}

generator client {
  provider = "prisma-client-js"
}

// ---------------- ENUMS ----------------

enum AuthProvider {
  local
  google
}

enum PermissionLevel {
  view_only
  full_access
  alerts_only
}

enum LinkStatus {
  pending
  active
  revoked
  expired
}

enum ProposedRole {
  patient
  caregiver
}

enum InvitationStatus {
  sent
  accepted
  declined
  expired
}

enum DoctorSource {
  ocr_scan
  manual
}

enum MedicineForm {
  tablet
  capsule
  syrup
  liquid
  injection
  ointment
  cream
  drops
  inhaler
  patch
  powder
  spray
  other
}

enum TrackingType {
  countable
  volume_ml
  weight_grams
}

enum MedicationStatus {
  active
```

```
  completed
  discontinued
}

enum AddedVia {
  manual
  ocr_scan
}

enum ScanStatus {
  processing
  completed
  failed
  needs_review
}

enum DoseStatus {
  pending
  taken
  skipped
  snoozed
  missed
}

enum NotificationType {
  reminder
  refill
  escalation
  weekly_summary
  system
}

enum NotificationChannel {
  email
  push
  sms
  in_app
}

enum NotificationStatus {
  sent
  delivered
  failed
  read
}

enum EscalationResolutionStatus {
  open
  resolved
  ignored
}

enum PeriodType {
  daily
  weekly
  monthly
}

enum ReportSentStatus {
  pending
  sent
  failed
}

enum SenderType {
  user
  ai
}

enum InsightType {
  adherence_pattern
  schedule_suggestion
}

enum SeverityLevel {
  mild
  moderate
  severe
}

enum Theme {
  light
```

```
  dark
}

// ---------------- MODELS ----------------

model User {
  id                Int       @id @default(autoincrement())
  fullName          String
  email             String    @unique
  phoneNumber       String?
  passwordHash      String?      // null for google-only accounts
  authProvider      AuthProvider @default(local)
  googleId          String?      @unique
  dateOfBirth       DateTime?
  gender            String?
  profilePictureUrl String?
  address           String?
  timezone          String?
  emailVerified     Boolean   @default(false)
  isActive          Boolean   @default(true)
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt
  lastLoginAt       DateTime?

  patientLinks              CaregiverPatientLink[]      @relation("PatientLinks")
  caregiverLinks            CaregiverPatientLink[]      @relation("CaregiverLinks")
  invitedLinks              CaregiverPatientLink[]      @relation("InvitedByLinks")
  sentInvitations           Invitation[]                @relation("InviterInvitations")
  doctors                   Doctor[]
  medications               Medication[]
  prescriptionScans         PrescriptionScan[]
  doseLogsLogged            DoseLog[]                   @relation("LoggedByUser")
  doseLogsPatient           DoseLog[]                   @relation("PatientDoseLogs")
  medicineInventory         MedicineInventory[]
  notifications             Notification[]
  escalationAlertsPatient   EscalationAlert[]           @relation("PatientEscalations")
  escalationAlertsCaregiver EscalationAlert[]           @relation("CaregiverEscalations")
  adherenceStats            AdherenceStat[]
  weeklyReportsCaregiver    WeeklySummaryReport[]       @relation("CaregiverReports")
  weeklyReportsPatient      WeeklySummaryReport[]       @relation("PatientReports")
  aiChatConversations       AiChatConversation[]
  aiInsights                AiInsight[]
  interactionFlags          MedicationInteractionFlag[]
  preferences               UserPreference?
  auditLogs                 AuditLog[]

  @@map("users")
}

model CaregiverPatientLink {
  id               Int             @id @default(autoincrement())
  patientId        Int
  caregiverId      Int
  relationshipType String
  permissionLevel  PermissionLevel
  status           LinkStatus      @default(pending)
  invitedByUserId  Int?
  linkedAt         DateTime?
  revokedAt        DateTime?
  createdAt        DateTime        @default(now())

  patient   User  @relation("PatientLinks", fields: [patientId], references: [id])
  caregiver User  @relation("CaregiverLinks", fields: [caregiverId], references: [id])
  invitedBy User? @relation("InvitedByLinks", fields: [invitedByUserId], references: [id])

  @@map("caregiver_patient_links")
}

model Invitation {
  id               Int              @id @default(autoincrement())
  inviterUserId    Int
  inviteeEmail     String
  relationshipType String
  proposedRole     ProposedRole
  inviteToken      String           @unique
  status           InvitationStatus  @default(sent)
  expiresAt        DateTime
  createdAt        DateTime          @default(now())
  respondedAt      DateTime?

  inviter User @relation("InviterInvitations", fields: [inviterUserId], references: [id])
  @@map("invitations")
```

```
}

model Doctor {
  id                 Int          @id @default(autoincrement())
  patientId          Int
  doctorName         String
  specialization     String?
  clinicHospitalName String?
  contactNumber      String?
  address            String?
  source             DoctorSource
  createdAt          DateTime     @default(now())
  updatedAt          DateTime     @updatedAt

  patient           User               @relation(fields: [patientId], references: [id])
  medications       Medication[]
  prescriptionScans PrescriptionScan[]

  @@map("doctors")
}

model Medication {
  id            Int              @id @default(autoincrement())
  patientId     Int
  doctorId      Int?
  medicineName  String
  genericName   String?
  medicineForm  MedicineForm
  trackingType  TrackingType
  dosageAmount  Decimal          @db.Decimal(6, 2)
  dosageUnit    String
  frequencyText String
  instructions  String?
  startDate     DateTime
  endDate       DateTime?
  status        MedicationStatus  @default(active)
  addedVia      AddedVia
  createdAt     DateTime          @default(now())
  updatedAt     DateTime          @updatedAt

  patient                  User                       @relation(fields: [patientId], references: [id])
  doctor                   Doctor?                    @relation(fields: [doctorId], references: [id])
  schedules                MedicationSchedule[]
  inventory                MedicineInventory[]
  prescriptionScans        PrescriptionScan[]
  aiChatConversations      AiChatConversation[]
  aiInsights               AiInsight[]
  interactionFlagsAsFirst  MedicationInteractionFlag[] @relation("MedicationOne")
  interactionFlagsAsSecond MedicationInteractionFlag[] @relation("MedicationTwo")

  @@map("medications")
}

model PrescriptionScan {
  id                Int         @id @default(autoincrement())
  patientId         Int
  medicationId      Int?
  doctorId          Int?
  imageUrl          String
  ocrRawText        String?
  ocrExtractedData  Json?
  ocrConfidenceScore Decimal?    @db.Decimal(5, 2)
  scanStatus        ScanStatus  @default(processing)
  uploadedAt        DateTime    @default(now())
  reviewedAt        DateTime?

  patient    User        @relation(fields: [patientId], references: [id])
  medication Medication? @relation(fields: [medicationId], references: [id])
  doctor     Doctor?     @relation(fields: [doctorId], references: [id])

  @@map("prescription_scans")
}

model MedicationSchedule {
  id             Int      @id @default(autoincrement())
  medicationId   Int
  timeOfDay      DateTime @db.Time()
  daysOfWeek     String
  doseAmount     Decimal  @db.Decimal(6, 2)
  isTaperSchedule Boolean  @default(false)
  taperDetails   Json?
  isActive       Boolean  @default(true)
```

```
  createdAt       DateTime @default(now())
  updatedAt       DateTime @updatedAt

  medication Medication @relation(fields: [medicationId], references: [id])
  doseLogs   DoseLog[]

  @@map("medication_schedules")
}

model DoseLog {
  id                  Int        @id @default(autoincrement())
  scheduleId          Int
  patientId           Int
  scheduledDatetime   DateTime
  actualActionDatetime DateTime?
  status              DoseStatus @default(pending)
  loggedByUserId      Int?
  notes               String?
  createdAt           DateTime   @default(now())

  schedule         MedicationSchedule @relation(fields: [scheduleId], references: [id])
  patient          User               @relation("PatientDoseLogs", fields: [patientId], references: [id])
  loggedBy         User?              @relation("LoggedByUser", fields: [loggedByUserId], references: [id])
  notifications    Notification[]
  escalationAlerts EscalationAlert[]

  @@map("dose_logs")
}

model MedicineInventory {
  id                     Int       @id @default(autoincrement())
  medicationId           Int
  patientId              Int
  quantityUnit           String
  quantityTotal          Decimal   @db.Decimal(8, 2)
  quantityRemaining      Decimal   @db.Decimal(8, 2)
  estimatedUsagePerDose Decimal?  @db.Decimal(6, 2)
  refillThreshold        Decimal   @db.Decimal(8, 2)
  refillAmount           Decimal   @db.Decimal(8, 2)
  estimatedRunoutDate    DateTime?
  lastRefillDate         DateTime?
  updatedAt              DateTime  @updatedAt

  medication Medication @relation(fields: [medicationId], references: [id])
  patient    User       @relation(fields: [patientId], references: [id])

  @@map("medicine_inventory")
}

model Notification {
  id               Int                @id @default(autoincrement())
  userId           Int
  relatedDoseLogId Int?
  notificationType NotificationType
  channel          NotificationChannel
  title            String
  message          String
  status           NotificationStatus  @default(sent)
  sentAt           DateTime            @default(now())
  readAt           DateTime?

  user    User     @relation(fields: [userId], references: [id])
  doseLog DoseLog? @relation(fields: [relatedDoseLogId], references: [id])

  @@map("notifications")
}

model EscalationAlert {
  id               Int                        @id @default(autoincrement())
  doseLogId        Int
  patientId        Int
  caregiverId      Int?
  escalationLevel  Int
  triggeredAt      DateTime                   @default(now())
  resolvedAt       DateTime?
  resolutionStatus EscalationResolutionStatus  @default(open)

  doseLog   DoseLog @relation(fields: [doseLogId], references: [id])
  patient   User    @relation("PatientEscalations", fields: [patientId], references: [id])
  caregiver User?   @relation("CaregiverEscalations", fields: [caregiverId], references: [id])

  @@map("escalation_alerts")
```

```
}

model AdherenceStat {
  id                  Int        @id @default(autoincrement())
  patientId           Int
  periodType          PeriodType
  periodStartDate     DateTime
  periodEndDate       DateTime
  totalScheduledDoses Int
  takenCount          Int
  missedCount         Int
  skippedCount        Int
  adherencePercentage Decimal    @db.Decimal(5, 2)
  generatedAt         DateTime   @default(now())

  patient User @relation(fields: [patientId], references: [id])

  @@map("adherence_stats")
}

model WeeklySummaryReport {
  id            Int              @id @default(autoincrement())
  caregiverId   Int
  patientId     Int
  weekStartDate DateTime
  weekEndDate   DateTime
  summaryText   String
  generatedAt   DateTime         @default(now())
  sentStatus    ReportSentStatus @default(pending)
  sentAt        DateTime?

  caregiver User @relation("CaregiverReports", fields: [caregiverId], references: [id])
  patient   User @relation("PatientReports", fields: [patientId], references: [id])

  @@map("weekly_summary_reports")
}

model AiChatConversation {
  id                  Int       @id @default(autoincrement())
  userId              Int
  relatedMedicationId Int?
  startedAt           DateTime  @default(now())
  endedAt             DateTime?

  user       User            @relation(fields: [userId], references: [id])
  medication Medication?     @relation(fields: [relatedMedicationId], references: [id])
  messages   AiChatMessage[]

  @@map("ai_chat_conversations")
}

model AiChatMessage {
  id              Int        @id @default(autoincrement())
  conversationId  Int
  senderType      SenderType
  messageText     String
  disclaimerShown Boolean    @default(false)
  createdAt       DateTime   @default(now())

  conversation AiChatConversation @relation(fields: [conversationId], references: [id])

  @@map("ai_chat_messages")
}

model AiInsight {
  id                  Int         @id @default(autoincrement())
  patientId           Int
  relatedMedicationId Int?
  insightType         InsightType
  insightText         String
  generatedAt         DateTime    @default(now())
  isDismissed         Boolean     @default(false)
  dismissedAt         DateTime?

  patient    User        @relation(fields: [patientId], references: [id])
  medication Medication? @relation(fields: [relatedMedicationId], references: [id])

  @@map("ai_insights")
}

model DrugInteraction {
  id              Int           @id @default(autoincrement())
```

```
  medicineNameA   String
  medicineNameB   String
  severityLevel   SeverityLevel
  description     String
  sourceReference String?

  flags MedicationInteractionFlag[]

  @@map("drug_interactions")
}

model MedicationInteractionFlag {
  id             Int       @id @default(autoincrement())
  patientId      Int
  medicationId1  Int
  medicationId2  Int
  interactionId  Int
  flaggedAt      DateTime  @default(now())
  acknowledged   Boolean   @default(false)
  acknowledgedAt DateTime?

  patient     User            @relation(fields: [patientId], references: [id])
  medication1 Medication      @relation("MedicationOne", fields: [medicationId1], references: [id])
  medication2 Medication      @relation("MedicationTwo", fields: [medicationId2], references: [id])
  interaction DrugInteraction @relation(fields: [interactionId], references: [id])

  @@map("medication_interaction_flags")
}

model UserPreference {
  id                          Int                @id @default(autoincrement())
  userId                      Int                @unique
  preferredNotificationChannel NotificationChannel
  language                    String             @default("en")
  theme                       Theme              @default(light)
  timezone                    String?
  updatedAt                   DateTime           @updatedAt

  user User @relation(fields: [userId], references: [id])

  @@map("user_preferences")
}

model AuditLog {
  id         Int      @id @default(autoincrement())
  userId     Int
  actionType String
  entityType String
  entityId   Int?
  ipAddress  String?
  timestamp  DateTime @default(now())

  user User @relation(fields: [userId], references: [id])

  @@map("audit_logs")
}
```

Recommended Project Structure

```
meditrack/
■■■ client/                        # React + Vite frontend
■   ■■■ src/
■   ■   ■■■ pages/                 # one folder per module (auth, dashboard, medications, ...)
■   ■   ■■■ components/
■   ■   ■■■ context/               # AuthContext, ActivePatientContext
■   ■   ■■■ api/                   # axios instance + per-resource API calls
■   ■   ■■■ App.jsx
■■■ server/                        # Node + Express backend
■   ■■■ prisma/
■   ■   ■■■ schema.prisma
■   ■■■ src/
■   ■   ■■■ routes/                # auth.routes.js, medications.routes.js, ...
■   ■   ■■■ controllers/
■   ■   ■■■ services/              # business logic (escalation.service.js, ocrClient.service.js)
■   ■   ■■■ middleware/            # auth.middleware.js, permission.middleware.js
■   ■   ■■■ jobs/                  # node-cron jobs (escalation, inventory, weekly reports)
■   ■   ■■■ sockets/               # socket.io setup + emitters
■   ■   ■■■ prismaClient.js
■   ■■■ .env                       # DATABASE_URL, JWT_SECRET, GEMINI_API_KEY, etc.
■■■ ocr-service/                   # Python FastAPI microservice
■   ■■■ main.py                    # loads PaddleOCR model ONCE at startup
■   ■■■ preprocessing.py           # grayscale > denoise > deskew > CLAHE > binarize
■   ■■■ paddle_engine.py           # PaddleOCR only (pinned config) + line assembly
■   ■■■ extraction.py              # regex extractor first, Gemini as fallback/normalizer
■   ■■■ tests/                     # benchmark test_set images as regression fixtures
■   ■■■ requirements.txt           # pinned: paddleocr 3.7.0, paddlepaddle 3.3.1, cv2 4.10.0.84
■■■ docker-compose.yml (optional)  # postgres + redis + services
```

Phase 0: Project Setup & Architecture (Week 1)

Foundational setup — no UI yet

Prisma models used: All models (verifying migration succeeded)

Tasks (in order):

- Initialize three folders: client/ (npm create vite@latest -- --template react), server/ (npm init -y, install express, prisma, @prisma/client, bcrypt, jsonwebtoken, ioredis, socket.io, node-cron, dotenv, cors), ocr-service/ (Python venv, install fastapi, uvicorn, opencv-python, pytesseract, paddleocr, easyocr, google-generativeai).
- Run npx prisma init inside server/, replace the generated schema.prisma with the complete schema provided in this document.
- Provision a PostgreSQL instance (local via Docker, or a managed free tier like Neon/Supabase/Railway) and set DATABASE_URL in server/.env.
- Run npx prisma migrate dev --name init to create every table, then npx prisma generate to produce the typed Prisma Client.
- Set up server/src/prismaClient.js exporting a single shared PrismaClient instance (never instantiate PrismaClient per-request).
- Set up Redis connection (local Docker or Upstash free tier) in server/src/config/redis.js.
- Create a health-check route on both server (GET /api/health) and ocr-service (GET /health).
- Confirm server/ can successfully call ocr-service/ over HTTP (simple ping test).

API Endpoints:

```
GET /api/health -> { status: 'ok' }
GET /health (ocr-service) -> { status: 'ok' }
```

Deliverable: All 20 tables exist in PostgreSQL (verified via npx prisma studio); all three services run locally and can reach each other.

Phase 1: Authentication & Account Management (Week 2)

Module 1, Pages 1-9: Sign Up, Login, Google Sign-In, Forgot/Reset Password, Email Verification, My Profile, Edit Profile, Account Settings, Notification Preferences

Prisma models used: User (with authProvider, googleId, nullable passwordHash), UserPreference

Tasks (in order):

- **Auth approach:** custom JWT + bcrypt (no Passport.js). Access token (short-lived, ~15 min) + refresh token (long-lived, ~7 days) stored in Redis keyed by userId so tokens can be revoked on logout. Google Sign-In is layered on top as an additional entry point that issues the SAME app JWTs.
- Build POST /api/auth/register: validate input, hash password with bcrypt (12 rounds), prisma.user.create({ authProvider: 'local' }), generate an email-verification token, send via Nodemailer, return 201.
- Build POST /api/auth/login: reject early if user.authProvider === 'google' and passwordHash is null (respond with a clear 'this account uses Google Sign-In' message); otherwise verify with bcrypt.compare, issue access + refresh tokens, store the refresh token in Redis with TTL.
- Build POST /api/auth/refresh: validate the refresh token against Redis, issue a new access token (rotate the refresh token for extra security).
- Build POST /api/auth/logout: delete the refresh token key from Redis.
- Build POST /api/auth/forgot-password: generate a reset token (crypto.randomBytes), store its hash in Redis with a short TTL, email the reset link. Skip/deny for google-only accounts.
- Build POST /api/auth/reset-password: verify token against Redis, prisma.user.update() with the new password hash.
- Build GET /api/auth/verify-email/:token: verify, prisma.user.update({ emailVerified: true }). Google accounts are created with emailVerified: true automatically, since Google already verified it.
- **Google Sign-In (frontend):** install @react-oauth/google, wrap App in <GoogleOAuthProvider clientId={VITE_GOOGLE_CLIENT_ID}>, render <GoogleLogin onSuccess={...}> on both the Login and Sign Up pages. The callback gives you credentialResponse.credential — a Google ID token (JWT).
- **Google Sign-In (backend):** install google-auth-library. Build POST /api/auth/google: call client.verifyIdToken({ idToken, audience: GOOGLE_CLIENT_ID }) to validate the token's signature, issuer, audience, and expiry — never trust the token payload without verification.
- In the same endpoint, extract sub (Google's stable user id), email, name, picture, email_verified from the verified payload.
- Account resolution logic (run inside prisma.$transaction): (a) if a user exists with that googleId, log them in; (b) else if a user exists with that email and authProvider 'local', link the accounts by setting googleId on the existing row (this prevents duplicate accounts for the same person); (c) else create a new user with authProvider: 'google', googleId, emailVerified: true, passwordHash: null, profilePictureUrl from picture.
- After resolution, issue the same access + refresh JWT pair as the local login path and store the refresh token in Redis — from this point on, the rest of the app is auth-method-agnostic.
- Create a default UserPreference row for every newly created user (both local and Google) so later notification logic never hits a null preferences record.
- Build GET /api/users/me, PATCH /api/users/me for profile view/edit. On the Account Settings page, show which provider the account uses and hide the 'change password' section for google-only accounts.
- Optional: build POST /api/auth/google/link and POST /api/auth/set-password so a Google user can add a password, or a local user can attach Google, from Account Settings.
- Build PATCH /api/users/me/preferences mapped to prisma.userPreference.upsert().
- Build the Express auth middleware (verifyJWT) that decodes the access token and attaches req.user; apply it to every protected route from this phase onward. It is identical for both auth methods.
- Env vars needed: JWT_ACCESS_SECRET, JWT_REFRESH_SECRET, GOOGLE_CLIENT_ID (backend), VITE_GOOGLE_CLIENT_ID (frontend). Register the app in Google Cloud Console -> APIs & Services -> Credentials -> OAuth 2.0 Client ID (Web application), adding your dev and prod origins to Authorized JavaScript origins.

API Endpoints:

```
POST /api/auth/register { fullName, email, phoneNumber, password }
POST /api/auth/login { email, password }
POST /api/auth/google { idToken } <- Google ID token from frontend
POST /api/auth/refresh { refreshToken }
POST /api/auth/logout
POST /api/auth/forgot-password { email }
POST /api/auth/reset-password { token, newPassword }
GET /api/auth/verify-email/:token
POST /api/auth/google/link { idToken } (optional, authed)
POST /api/auth/set-password { newPassword } (optional, authed)
GET /api/users/me
PATCH /api/users/me { fullName, phoneNumber, address, profilePictureUrl }
PATCH /api/users/me/preferences { preferredNotificationChannel, language, theme, timezone }
```

Deliverable: Full auth flow testable end-to-end via both paths: email/password register -> verify -> login, AND one-click Google Sign-In — both issuing identical app JWTs, with existing email accounts correctly linked rather than duplicated.

Phase 2: Role & Relationship Management (Week 3)

Module 1, Pages 10-13: My Relationships, Invite Caregiver/Patient, Accept/Decline Invitation, Manage Permissions

Prisma models used: CaregiverPatientLink, Invitation, User

Tasks (in order):

- Build GET /api/relationships: query prisma.caregiverPatientLink.findMany() with OR on patientId/caregiverId = req.user.id, include both patient and caregiver user objects, label each row's perspective in the response.
- Build POST /api/invitations: generate a secure token (crypto.randomBytes(32).toString('hex')), prisma.invitation.create(), email the invitee via Nodemailer with a link containing the token.
- Build GET /api/invitations/:token: look up the invitation, return relationshipType/proposedRole/inviter info for the accept/decline UI.
- Build POST /api/invitations/:token/accept: if the invitee isn't registered, redirect to signup first (carry the token through); once authenticated, prisma.$transaction to create the caregiverPatientLink (status: active, linkedAt: now) and update the invitation (status: accepted, respondedAt: now).
- Build POST /api/invitations/:token/decline: update invitation status to declined.
- Build PATCH /api/relationships/:linkId/permissions: update permissionLevel or status (revoked); only the patient side of the link may call this — enforce with an ownership check in the controller.
- Write a checkPermission(requiredLevel) middleware that loads the relevant caregiverPatientLink and rejects the request if permissionLevel is insufficient. Use this middleware on every caregiver-facing route built in later phases.

API Endpoints:

```
GET /api/relationships
POST /api/invitations { inviteeEmail, relationshipType, proposedRole }
GET /api/invitations/:token
POST /api/invitations/:token/accept
POST /api/invitations/:token/decline
PATCH /api/relationships/:linkId/permissions { permissionLevel } or { status: 'revoked' }
```

Deliverable: A patient can invite a caregiver by email, the caregiver accepts (registering first if needed), and permission level correctly gates later API access.

Phase 3: Dashboards (Week 4)

Module 2, Pages 14-16: Patient Home Dashboard, Caregiver Home Dashboard, Multi-Patient Overview

Prisma models used: DoseLog, AdherenceStat, MedicineInventory, CaregiverPatientLink

Tasks (in order):

- Build GET /api/dashboard/patient: aggregate today's dose_logs (via schedule -> medication -> patientId), latest AdherenceStat (periodType: daily), and MedicineInventory rows where estimatedRunoutDate is within 7 days.
- Build GET /api/dashboard/caregiver/:patientId: verify an active caregiverPatientLink exists via the checkPermission middleware, then return the same shape as the patient dashboard but read-only, with fields filtered based on permissionLevel (e.g. alerts_only omits full adherence detail).
- Build GET /api/dashboard/my-patients: prisma.caregiverPatientLink.findMany({ where: { caregiverId, status: 'active' }, include: { patient: true } }) for the Multi-Patient Overview cards.
- Frontend: build an ActivePatientContext (React Context) so the caregiver's selected patient persists across the caregiver-side pages built in later phases.

API Endpoints:

```
GET /api/dashboard/patient
GET /api/dashboard/caregiver/:patientId
GET /api/dashboard/my-patients
```

Deliverable: Patients and caregivers see correctly scoped, live dashboards; caregivers can switch between multiple linked patients.

Phase 4: Prescription Scanning & Python OCR Microservice (Weeks 5-6)

Module 3, Pages 17-18 + OCR task breakdown. ENGINE ALREADY SELECTED: PaddleOCR (see benchmark decision record above) — do NOT implement engine-selection logic or re-run the comparison.

Prisma models used: PrescriptionScan, Doctor, Medication

Tasks (in order):

- 1) Frontend: file input on the Upload/Scan Prescription page, POST the image as multipart/form-data to Express.
- 2) Express: upload the image to S3/Blob Storage, prisma.prescriptionScan.create({ scanStatus: 'processing', imageUrl }), then POST the image URL to ocr-service (http://ocr-service:8001/extract).
- 3) Python preprocessing (preprocessing.py) — reuse the exact pipeline already validated in the benchmark: grayscale -> denoise -> pixel-based deskew (no answer-key angle) -> CLAHE -> adaptive binarization. Keep this as a single reusable function so the production path and any future re-benchmark receive identical pixels. Measured shared preprocessing cost: ~0.56 s/image.
- 4) Python OCR (ocr_engines/paddle.py) — PaddleOCR only, pinned to the benchmarked configuration: PP-OCRv5_mobile_det + en_PP-OCRv5_mobile_rec, document/orientation models disabled, MKLDNN disabled, CPU with 4 compute threads. Pin paddleocr 3.7.0 / paddlepaddle 3.3.1 in requirements.txt, and align opencv-contrib-python with opencv-python-headless at the same version (4.10.0.84) — mismatched cv2 installs were an actual failure encountered during benchmarking.
- 5) Python: group returned bounding boxes into lines in visual order WITHOUT altering recognized tokens (same line-assembly logic as the benchmark), producing the raw transcript.
- 6) Load the PaddleOCR model ONCE at FastAPI startup (module-level singleton or a lifespan handler), never per-request — model initialization measured ~5.9 s, versus ~6.1 s steady-state inference per image. Loading per request would roughly double latency.
- 7) Python extraction (extraction.py): run the regex/heuristic field extractor first (the benchmark's extractor already recovers doctor, degree, clinic, phone, address, patient name/age/gender, and per medicine name/form/dose

amount/unit/frequency/duration at 99.31% field score on this workload), then send the raw OCR text to Gemini ONLY to normalize/fill fields the regex pass could not resolve and to coerce output into the final JSON contract. Regex-first keeps the deterministic, already-measured path primary and uses the LLM as a fallback, not the sole parser.

- 8) JSON contract returned to Express: { doctor: { doctorName, specialization, clinicHospitalName, contactNumber, address }, medicines: [{ medicineName, genericName, medicineForm, dosageAmount, dosageUnit, frequencyText, instructions }], ocrRawText, ocrConfidenceScore }. medicineForm MUST be restricted to the MedicineForm enum in schema.prisma.
- 9) Confidence handling: PaddleOCR's native mean token confidence measured ~0.989 on this set. Store it in ocrConfidenceScore, and set scanStatus to 'needs_review' always (never auto-commit), but surface a visible low-confidence warning on the review page when it falls below a threshold (~0.95 is a reasonable starting line given the measured baseline).
- 10) Express: prisma.prescriptionScan.update({ ocrRawText, ocrExtractedData, ocrConfidenceScore, scanStatus: 'needs_review' }); emit a Socket.io event to the uploading user so the frontend leaves the loading state without polling. Because inference is ~6 s CPU, treat this as async from the start — never block the HTTP upload response on OCR completion.
- 11) Frontend: OCR Review & Confirm page renders the JSON as an editable form (one section per medicine + one doctor section). Human confirmation is mandatory — the benchmark was run on synthetic printed slips, not real photographed or handwritten prescriptions, so field accuracy on real-world input is not established.
- 12) Express confirm endpoint: prisma.$transaction to upsert the Doctor row (source: 'ocr_scan'), create each Medication row (addedVia: 'ocr_scan'), link them back to the prescriptionScan (medicationId, doctorId), and set scanStatus: 'completed', reviewedAt: now.
- 13) Keep the benchmark's test_set images as regression fixtures in ocr-service/tests/ so future preprocessing changes can be checked against known-good CER, rather than re-running a full engine comparison.

API Endpoints:

```
POST /api/prescription-scans multipart image upload
GET /api/prescription-scans/:id poll status / fetch extracted JSON
POST /api/prescription-scans/:id/confirm { doctor: {...}, medicines: [{...}] }
POST http://ocr-service:8001/extract { imageUrl } -> { ocrRawText, ocrExtractedData,
ocrConfidenceScore } (internal, Express -> Python only)
Socket event (server -> client): 'scan-complete' { scanId, scanStatus }
```

Deliverable: Uploading a real prescription photo produces a reviewable, editable structured form via the PaddleOCR pipeline; confirming it creates real Medication and Doctor rows in Postgres.

Phase 5: Medication & Doctor Management (Week 7)

Module 3, Pages 19-23: My Medications List, Add Medication (Manual), Edit Medication, Medication Detail, My Doctors List, Add/Edit Doctor

Prisma models used: Medication, Doctor, MedicationSchedule, MedicineInventory, DoseLog, AiInsight

Tasks (in order):

- Build full CRUD for Medication: GET list (filter by status), POST create (addedVia: 'manual'), PATCH update, DELETE (or soft-delete via status: 'discontinued').
- Frontend: medicineForm dropdown drives a client-side lookup table mapping form -> trackingType (tablet/capsule -> countable; syrup/liquid -> volume_ml; ointment/cream/powder -> weight_grams) so trackingType is always set consistently with the Prisma enum.
- Build GET /api/medications/:id (Medication Detail): prisma.medication.findUnique with include for schedules, inventory, and a recent slice of doseLogs, plus a separate query for aiInsights where relatedMedicationId matches.
- Build full CRUD for Doctor scoped to patientId; source is set to 'manual' unless the record originated from Phase 4.

API Endpoints:

```
GET /api/medications?status=active
```

```
POST /api/medications { medicineName, genericName, medicineForm, dosageAmount, dosageUnit, frequencyText, instructions, startDate, endDate, doctorId }
GET /api/medications/:id
PATCH /api/medications/:id
DELETE /api/medications/:id
GET /api/doctors
POST /api/doctors { doctorName, specialization, clinicHospitalName, contactNumber, address }
PATCH /api/doctors/:id
```

Deliverable: Full medication and doctor management working independent of OCR.

Phase 6: Schedule Management & Inventory (Week 8)

Module 3, Pages 24-27: Reminder Schedule Setup, Custom/Taper Schedule Editor, Medicine Inventory, Refill Reminder Settings

Prisma models used: MedicationSchedule, MedicineInventory, Notification

Tasks (in order):

- Build POST /api/medications/:id/schedules/generate: send frequencyText to Gemini, ask for a JSON array of { timeOfDay, daysOfWeek, doseAmount }; prisma.medicationSchedule.createMany() with the result; return the created rows for the user to edit before saving.
- Build standard CRUD on MedicationSchedule for manual overrides; for taper schedules, accept isTaperSchedule: true and a taperDetails JSON array of { fromDate, toDate, doseAmount } steps.
- Build POST /api/medications/:id/inventory: create the MedicineInventory row on first setup (quantityUnit derived from trackingType, quantityTotal, refillThreshold, refillAmount).
- Build a node-cron job (server/src/jobs/inventoryJob.js, runs daily at a fixed time): for each active inventory row, compute estimatedRunoutDate from quantityRemaining / (estimatedUsagePerDose * doses per day derived from schedules); if quantityRemaining < refillThreshold, prisma.notification.create({ notificationType: 'refill', ... }).
- Wire dose-log 'taken' actions (built in Phase 7) to decrement quantityRemaining by estimatedUsagePerDose (or 1 unit for countable forms) via a shared service function so inventory and adherence logic don't duplicate this decrement independently.

API Endpoints:

```
POST /api/medications/:id/schedules/generate { frequencyText }
GET /api/medications/:id/schedules
PATCH /api/schedules/:id { timeOfDay, daysOfWeek, doseAmount, isTaperSchedule, taperDetails }
POST /api/medications/:id/inventory { quantityUnit, quantityTotal, refillThreshold, refillAmount, estimatedUsagePerDose }
GET /api/medications/:id/inventory
```

Deliverable: Adding a medicine auto-generates an editable schedule and inventory countdown; taper dosing and refill thresholds work correctly and trigger real notifications.

Phase 7: Adherence Tracking (Week 9)

Module 4, Pages 28-31: Today's Doses, Dose Log History, Adherence Calendar Heatmap, Adherence Streaks & Stats

Prisma models used: DoseLog, MedicationSchedule, AdherenceStat, MedicineInventory

Tasks (in order):

- Build a node-cron job (midnight, per user timezone if feasible) that generates today's DoseLog rows (status: pending) from every active MedicationSchedule matching today's daysOfWeek.

- Build PATCH /api/dose-logs/:id: sets status (taken/skipped/snoozed), actualActionDatetime: now, loggedByUserId: req.user.id; if status === 'taken', call the inventory-decrement service function from Phase 6.
- Build GET /api/dose-logs?from=&to=&medicationId= for history, filterable.
- Build a nightly node-cron job that aggregates the day's DoseLog rows per patient into an AdherenceStat row (periodType: 'daily'); a weekly job rolls dailies into a 'weekly' row.
- Build GET /api/adherence/heatmap?months=3: return date -> adherencePercentage pairs from AdherenceStat for the calendar heatmap component.
- Build GET /api/adherence/stats: current streak (consecutive days with 100% adherence, computed in the service layer by walking AdherenceStat rows backward from today), best streak, and rolling 7/30-day percentage.

API Endpoints:

```
GET /api/dose-logs/today
PATCH /api/dose-logs/:id { status, notes }
GET /api/dose-logs?from=&to=&medicationId=
GET /api/adherence/heatmap?months=3
GET /api/adherence/stats
```

Deliverable: Real-time dose logging with an accurate, pre-aggregated adherence history, heatmap, and streak calculation.

Phase 8: Escalation & Real-Time Alerts (Week 10)

Module 4, Pages 32-33 + core of Module 5

Prisma models used: DoseLog, EscalationAlert, Notification, CaregiverPatientLink

Tasks (in order):

- Build a node-cron job (runs every few minutes) that finds DoseLog rows still status: 'pending' whose scheduledDatetime + gracePeriod has passed, with no existing open EscalationAlert.
- Level 1: prisma.escalationAlert.create({ escalationLevel: 1, caregiverId: null }), send a second push reminder Notification to the patient.
- Level 2 (after a second grace window with the dose still pending): for every active caregiverPatientLink where permissionLevel is full_access or alerts_only, create an escalationLevel: 2 EscalationAlert row per caregiver, create a Notification row per caregiver, and emit a Socket.io event to that caregiver's room (io.to('user:' + caregiverId).emit('missed-dose-alert', payload)).
- Server: on socket connection, authenticate the socket via the JWT and join the room 'user:' + userId (server/src/sockets/index.js).
- Build PATCH /api/escalations/:id/resolve for when a dose is logged after the fact (sets resolvedAt, resolutionStatus: 'resolved').
- Build GET/PATCH /api/escalation-settings for grace-period length and which linked caregivers are included in the chain (store as a JSON field on CaregiverPatientLink or a small config table if you want per-link granularity — simplest MVP: a fixed grace period in .env, escalate to all full_access caregivers).

API Endpoints:

```
GET /api/escalations/active
PATCH /api/escalations/:id/resolve
GET /api/escalation-settings
PATCH /api/escalation-settings { gracePeriodMinutes }
Socket event (server -> client): 'missed-dose-alert' { patientId, medicineName, scheduledDatetime }
```

Deliverable: A missed dose escalates automatically and a linked caregiver receives a real-time Socket.io alert plus a durable Notification row, within seconds during a live demo.

Phase 9: Caregiver Mode - Multi-Patient & Reports (Week 11)

Module 5, Pages 34-39: Patient Adherence Overview, Real-Time Alert Feed, Linked Patients List, Switch Patient View, Weekly Summary Reports, Report Archive

Prisma models used: CaregiverPatientLink, EscalationAlert, WeeklySummaryReport, AdherenceStat, DoseLog

Tasks (in order):

- Build GET /api/caregiver/patients (reuses Phase 3's my-patients query if not already generalized).
- Build GET /api/caregiver/alerts-feed: prisma.escalationAlert.findMany({ where: { caregiverId: req.user.id }, orderBy: { triggeredAt: 'desc' } }) plus live updates via the same Socket.io room joined in Phase 8.
- Build a weekly node-cron job (Sunday night): for each active caregiverPatientLink, gather the past week's AdherenceStat + DoseLog rows for that patient, send them to Gemini asking for a short plain-language summary, prisma.weeklySummaryReport.create(), then send via the caregiver's preferredNotificationChannel and mark sentStatus.
- Build GET /api/reports/weekly?patientId= and GET /api/reports/weekly/:id for the Report Archive page.

API Endpoints:

```
GET /api/caregiver/patients
GET /api/caregiver/alerts-feed
GET /api/reports/weekly?patientId=
GET /api/reports/weekly/:id
```

Deliverable: A caregiver with multiple linked patients can switch between them, see a live alert feed, and receive automated weekly summaries.

Phase 10: AI Assistant - Chat, Insights & Interaction Flags (Week 12)

Module 6, Pages 40-43: AI Chat Interface, Chat History, Adherence Insights, Schedule Conflict Alerts

Prisma models used: AiChatConversation, AiChatMessage, AiInsight, DrugInteraction, MedicationInteractionFlag

Tasks (in order):

- Build POST /api/chat/conversations to start a conversation (optionally scoped to relatedMedicationId).
- Build POST /api/chat/conversations/:id/messages: save the user message (prisma.aiChatMessage.create({ senderType: 'user' })), call Gemini with a system prompt that forces a 'this is not medical advice, consult your doctor' framing on every response, save the AI reply with disclaimerShown: true always set true regardless of question content.
- Build GET /api/chat/conversations for history, GET /api/chat/conversations/:id/messages for a thread.
- Build a scheduled job (weekly or on dashboard load, cached) that analyzes recent DoseLog patterns per patient, asks Gemini for a one-line behavioral insight, prisma.aiInsight.create({ insightType: 'adherence_pattern' }).
- Seed the DrugInteraction table with a curated static dataset (30-50 well-known interaction pairs, loaded via a Prisma seed script: prisma/seed.js run with npx prisma db seed) — never let the LLM generate these rows.
- Build a service function (runs whenever a new Medication is added) that checks the patient's other active medications against DrugInteraction (case-insensitive match on medicineNameA/B in either order) and creates MedicationInteractionFlag rows for matches.
- Build GET /api/insights (adherence insights + interaction flags combined for the Insights page), PATCH /api/insights/:id/dismiss.

API Endpoints:

```
POST /api/chat/conversations { relatedMedicationId? }
POST /api/chat/conversations/:id/messages { messageText }
GET /api/chat/conversations
GET /api/chat/conversations/:id/messages
```

```
GET /api/insights
PATCH /api/insights/:id/dismiss
```

Deliverable: Working chat with enforced disclaimers, AI-generated weekly insights, and correctly flagged medication conflicts sourced from the static dataset only.

Phase 11: Notifications Center & Compliance Pages (Week 13)

Modules 7 & 8, Pages 44-49: Notifications Inbox, Notification Channel Settings, FAQ, Contact Support, Terms of Service, Privacy Policy, Medical Disclaimer

Prisma models used: Notification, AuditLog (+ optional SupportTicket model if added)

Tasks (in order):

- Build GET /api/notifications (unified inbox across all notificationType values), PATCH /api/notifications/:id/read.
- Channel settings reuse the UserPreference endpoint built in Phase 1 — just add a settings-page UI if not already exposed there.
- Build static content pages (FAQ, Terms of Service, Privacy Policy, Medical Disclaimer) as simple React routes with markdown or plain content — write a genuine privacy section describing exactly what health data is stored and who can access it (patients, linked caregivers per permissionLevel).
- Build a small AuditLog write helper (server/src/services/audit.service.js) and call it from: login (record actionType as 'login_local' or 'login_google' so the audit trail shows which method was used), account linking, any caregiver read of a patient's health data, and permission-level changes on a CaregiverPatientLink. Build GET /api/audit-logs?userId= for admin/debugging visibility.
- Build a basic Contact Support form: POST /api/support-tickets storing to a lightweight table or just emailing the support inbox for the MVP (a dedicated SupportTicket model can be added to schema.prisma later if needed).

API Endpoints:

```
GET /api/notifications
PATCH /api/notifications/:id/read
GET /api/audit-logs?userId=
POST /api/support-tickets { subject, message }
```

Deliverable: A complete notifications experience and compliance section, with audit logging visibly recording sensitive actions.

Phase 12: Testing, Deployment & Demo Prep (Week 14)

Final polish and packaging

Prisma models used: All models (final integration test pass)

Tasks (in order):

- Write Jest tests for the highest-risk logic: escalation timing (Phase 8), OCR-JSON-to-Prisma mapping (Phase 4 confirm endpoint), adherence percentage calculation (Phase 7), inventory decrement math (Phase 6), and the Google account-resolution branches from Phase 1 (existing googleId / existing email to link / brand-new user — mock verifyIdToken so tests don't hit Google).
- Deploy PostgreSQL on a managed provider (Neon, Supabase, or Railway Postgres — all have generous free tiers and work seamlessly with Prisma's connection pooling).
- Deploy server/ and ocr-service/ on Render or Railway, on the same private network so Express can reach the Python service by internal hostname; run npx prisma migrate deploy as part of the server build step (not migrate dev, which is for local development only).
- Deploy client/ on Vercel or Netlify; deploy Redis on Upstash.

- Add the production frontend URL to Authorized JavaScript origins in the Google Cloud Console OAuth client, and set GOOGLE_CLIENT_ID / VITE_GOOGLE_CLIENT_ID in the production environment — Google Sign-In silently fails in production if the deployed origin isn't whitelisted.
- Record a demo video: scan prescription -> review OCR output -> schedule generated -> log a dose -> simulate a missed dose -> caregiver gets a real-time alert -> ask the AI chat a question and see the disclaimer.
- Write the README documenting: the three-service architecture, why Prisma + PostgreSQL was chosen, the OCR engine benchmark with the real measured numbers (PaddleOCR 0.10% CER / 99.31% field score vs Tesseract 1.54% / 98.16% vs EasyOCR 2.02% / 86.68%, across 15 synthetic degraded prescriptions) INCLUDING its stated limitations, and the static-dataset-over-LLM safety decision for drug interactions.

Deliverable: A fully deployed, demoable web app on PostgreSQL + Prisma, plus a portfolio-ready README and video.

Timeline Summary

Week	Phase	Focus
1	Phase 0	Setup: 3 services, Prisma schema, Postgres migration
2	Phase 1	Auth & account management
3	Phase 2	Role & relationship management
4	Phase 3	Dashboards
5-6	Phase 4	Prescription scanning & Python OCR pipeline
7	Phase 5	Medication & doctor management
8	Phase 6	Schedule management & inventory
9	Phase 7	Adherence tracking
10	Phase 8	Escalation & real-time alerts
11	Phase 9	Caregiver mode: multi-patient & reports
12	Phase 10	AI assistant: chat, insights, interactions
13	Phase 11	Notifications center & compliance
14	Phase 12	Testing, deployment, demo prep